Multi-Objective AutoML for Interpretable Model Selection in Tabular Classification Problems

Munro Ross

A thesis submitted for the degree of MSc Data Science

Department of Computing

University of Essex

February 2026

Table of Contents

List of Tables 3
List of Figures 4
Section 1: Introduction 5
AutoML 5
Interpretability and Explainability 5
Gaps in the field 6
Research Problem Statement 7
Research questions 7
Aim 7
Objectives 7
Dissertation Overview 8
Section 2: Literature Review 9
Background 9
Supervised classification and Evaluation 10
Interpretability: Concepts and Definitions 11
Interpretability Methods and Metrics 13
Methods 13
Metrics 14
AutoML and Model Selection 16
Interpretability in AutoML and Research Gaps 17
Section 3: Methodology 20
Research Design 20
Datasets 21
Preprocessing pipeline 23
Model selection 25
Performance Evaluation 26
SHAP Stability 27
SHAP Sparsity 29
Multi-Objective Ranking 30
Reproducibility and implementation 31
Section 4: Results 32
Experimental Overview 32
Dataset Level Results 32
Sensitivity to performance ratio (α) 40

Section 5: Discussion 42
Summary of key findings 42
Performance vs Interpretability Trade-off 42
Role of Stability and Sparsity 43
User Preference α Sensitivity 44
Implications for AutoML 44
Limitations 45
Section 6: Conclusion 46
References 48
Appendices 53
A: Experiments and Code 53

List of Tables

Table 1: Performance and interpretability metrics for Telco Customer Churn dataset. 33
Table 2: Performance and interpretability metrics for Breast Cancer Wisconsin (Diagnostic) dataset. 34
Table 3: Performance and interpretability metrics for Statlog (German Credit Data). 36
Table 4: Performance and interpretability metrics for Wine Quality dataset. 37
Table 5: Performance and interpretability metrics for Glass Identification dataset. 39

List of Figures

Figure 1: Taxonomy of methods by Molnar (2025). 12
Figure 2: Section of code showing the pipeline steps. 23
Figure 3: GroupFeatures class for feature handling. 24
Figure 4: Column transformer for preprocessing of specific feature types. 25
Figure 5: Dictionary of models for each experimental run. 26
Figure 6: Function for model evaluation on test set. 27
Figure 7: Function for implementing multi-objective scoring. 31
Figure 8: Composite interpretability of Stability and Sparsity compared with performance metric ROC AUC for the Telco Customer Churn dataset, interpretability ratio = 0.5. 33
Figure 9: Comparison of Stability and Sparsity for the Telco Customer Churn dataset. 34
Figure 10: Composite interpretability of Stability and Sparsity compared with performance metric ROC AUC for the Breast Cancer Wisconsin dataset, interpretability ratio = 0.5. 35
Figure 11: Comparison of Stability and Sparsity for the Breast Cancer Wisconsin dataset. ... 35
Figure 12: Composite interpretability of Stability and Sparsity compared with performance metric ROC AUC for the Statlog dataset, interpretability ratio = 0.5. 36
Figure 13: Comparison of Stability and Sparsity for the Statlog dataset. 37
Figure 14: Composite interpretability of Stability and Sparsity compared with performance metric ROC AUC for the Wine Quality dataset, interpretability ratio = 0.5. 38
Figure 15: Comparison of Stability and Sparsity for the Wine Quality dataset. 38
Figure 16: Composite interpretability of Stability and Sparsity compared with performance metric ROC AUC for the Glass Identification dataset, interpretability ratio = 0.5 39
Figure 17: Comparison of Stability and Sparsity for the Glass Identification dataset. 40
Figure 18: Sensitivity of multi-objective ranking score to performance ratio α for Wine Quality Dataset. 40

Section 1: Introduction

AutoML

Machine learning is becoming more prevalent and popular in daily life for many people, from self-driving cars to LLM's such as ChatGPT. It is also being used even more in professional industries such as Healthcare, Finance, Manufacturing and Cybersecurity. Implementing Machine learning into a business or application requires the building of a specialized pipeline, this would involve using a team of Data Science and Domain experts, take a lot of time and resources making it an expensive pursuit (Zöller and Huber, 2019).

Automated Machine Learning (AutoML) aims to simplify the process of building models by automating the processes in the pipeline such as data preprocessing, algorithm selection, and hyperparameter optimisation to produce comparable models to those of the experts without as much manual intervention. This automation helps boost productivity of existing experts, reduce the costs of model development, and democratise access to machine learning for non-experts (He, Zhao and Chu, 2021; Feng et al., 2024) .

AutoML systems typically focus on optimising and evaluating models using cross-validation and select the best hyperparameter configuration that maximises predefined predictive performance metrics, such as accuracy or ROC AUC. As a result of this, predictive performance becomes the primary objective to guide model selection, with these pipelines only outputting models based on this. To increase predictive performance as much as possible, AutoML tools usually utilise techniques such as ensembling and stacking of models, increasing the adoption of complex model structures, reducing model transparency and complicating interpretability (Zöller and Huber, 2019; Schneider, Bernd Bischl and Thomas, 2023).

Interpretability and Explainability

As machine learning systems are increasingly deployed into high-stakes domains such as healthcare, finance and criminal justice to name a few, there is a corresponding demand for more interpretable and transparent model. In regulated and socially sensitive settings there is a requirement by stakeholders that in addition to accurate predictions, explanations to justify and contextualise the decisions are needed. This is enforced by regulatory bodies such as The European Data Protection Regulation, which states that data processing should be lawful, fair and transparent to the data subject (GDPR, 2018).

Doshi-Velez and Kim (2017) argue that interpretability is essential for ascertaining trust, fairness, privacy, reliability and more desiderata in automated decision-making domains. Similarly, Rudin (2019) argues that reliance on black box models can be problematic in critical applications, advocating for the use of more inherently interpretable models overall.

Interpretability in machine learning has two main approaches to address it, using inherently interpretable models and post-hoc explanations. Inherently interpretable models, such as linear models and small decision trees, are transparent by design in their construction. The structure allows a user to examine the internals of the model and observe any coefficients or learned parameters. In contrast, post-hoc methods do not give insights into the internal mechanisms of the model, they aim to provide insights into the behaviour of the model and its decision logic. The nature of inherent interpretability makes it model specific while post-hoc methods can be both model specific and model agnostic depending on the method (Molnar, 2025).

Gaps in the field

While AutoML has advance automated model development, they remain largely performance-centric in their selection criteria with not many options for model interpretability considerations. Normally a model is selected based on optimisation of performance metrics, such as accuracy or ROC AUC, while interpretability considerations are assessed post hoc and outside of the AutoML pipeline.

Most explanation techniques do not provide an overall global quantitative metric for interpretability, techniques such as SHAP and LIME provide quantitative measure of feature attribution, however these are normally limited to local prediction-based explanations. In addition, even more explanation techniques provide qualitative results over a more measurable quantitative result. Particularly, there is a lack of lightweight, model-agnostic approaches that incorporate predictive performance and quantitative interpretability measures into a unified model ranking formula. Lastly, it remains unclear whether a performance-interpretability trade-off is shows consistent patterns across different tabular classification datasets, or whether this is dataset dependent.

This dissertation addresses these gaps by proposing and evaluating a multi-objective ranking framework that integrates SHAP-based interpretability metrics directly into AutoML model selection. Systematically analysing the trade-offs between performance and interpretability, as

well as preference-sensitive behaviour across multiple datasets. The study investigates whether interpretability-aware ranking meaningfully alters model selection outcomes.

Research Problem Statement

Many AutoML tools focus on maximising predictive performance, such as accuracy or f1-score, when ranking and prioritising model creation and can lead to the selection of more complex and less explainable models. This is also made more complicated with the fact that automating the creation of the model adds further opacity to the decision-making process (Zöller and Huber, 2019).

While post-hoc auditing of models is viable, another solution would be to have an AutoML framework that already considers interpretability alongside predictive performance when ranking the best models to use.

Research questions

1. Can SHAP-based interpretability metrics be integrated into an AutoML model ranking framework for tabular classification?
2. What is the relationship between predictive performance and interpretability, and how does incorporating interpretability affect model ranking?

Aim

To develop and evaluate a lightweight multi-objective AutoML pipeline that integrates quantitative interpretability metrics alongside predictive performance into model selection for tabular classification.

Objectives

- Implement a multi-objective AutoML pipeline integrating quantitative model-agnostic interpretability metrics into model ranking.
- Evaluate the AutoML pipeline across multiple tabular classification datasets with different characteristics to analyse performance-interpretability behaviour.
- Analyse whether a trade-off is present between predictive performance and interpretability.

Dissertation Overview

This dissertation is structured as follows. Chapter 2 reviews existing literature on the topics of AutoML, explainable AI (Interpretability), and the relationship between predictive performance and model interpretability. Chapter 3 presents the proposed methodology, including data preprocessing, model selection procedures, interpretability metrics and model ranking. Chapter 4 reports experimental results across five tabular datasets and presents visualisations. Chapter 5 discusses the findings and relates to existing literature. Chapter 6 Concludes the dissertation and outlines avenues for future work.

Section 2: Literature Review

Background

What is AutoML and what is it useful for?

Automated Machine Learning (AutoML) simplifies the development of machine learning models by automating the steps in the pipeline that would normally require some manual human intervention. These steps can include data preprocessing, feature engineering, model selection, hyperparameter tuning and model evaluation (Feng et al., 2024).

Zöller and Huber (2019) explains that AutoML can be beneficial for current Machine Learning (ML) experts in terms of increasing workflow productivity and efficiency by automating tedious tasks, as well as enabling domain experts to utilise ML pipelines without the need for a data scientist, democratizing machine learning and AI.

Explainability, Interpretability and why does it matter?

Explainability and Interpretability are terms often used interchangeably in literature. Biran and Cotton (2017) define a system as being interpretable "if its operations can be understood by a human", this is further explored by Miller (2019) who equates the two terms but states that an explanation is just one mode in which understanding can be gained.

Molnar (2025) also examines the definitions of Interpretability and Explainability in literature and concludes that the "terms are so fuzzy it is pragmatic to see them as an umbrella term" that captures the same thing. For the purpose of this project, the terms will be treated equally as adopted by Miller.

While not all applications of machine learning require interpretability, many regulated industries such as healthcare, finance and criminal justice are utilising ML for high-stakes predictions that can influence decisions made, and drastically impact people's lives (Carvalho, Pereira and Cardoso, 2019). The increasingly widespread and diverse use of AI and ML leads to the need for explanations of a model's reasoning to ensure the following desiderata: fairness, privacy, reliability, robustness, causality, usability and trust (Doshi-Velez and Kim, 2017).

Statement of Angle.

This review explores how interpretability can be quantified and incorporated into the model selection process of AutoML, with a focus on tabular classification.

Guiding Questions.

Here is a list of guiding questions that helped in reviewing of existing literature:

- How has interpretability been defined and justified in literature?
- What does existing literature say about the relationship between interpretability and performance in machine learning models?
- How can model interpretability be quantified and incorporated into AutoML?
- How does AutoML currently select models?
- What existing work has been done in combining AutoML and interpretability?

Map of the review.

The review is organised into the following themes: interpretability concepts and methodologies, model performance vs interpretability, current AutoML strategies and frameworks and implications for the project.

Supervised classification and Evaluation

Supervised classification and evaluation

Supervised classification is a key machine learning paradigm that involves training a model on labelled data to predict predefined discrete classes for previously unseen instances. Some examples of applications include customer churn prediction, medical diagnosis, and fraud detection. To ensure that the trained model can generalise well, it is normally evaluated on unseen data that are partitioned from the dataset before training begins, these partitions are referred to as the training and test sets.

Cross-validation is a statistical resampling method that can be used to train the model and evaluate its generalisation performance in a more stable way than using a train-test split of the data. It involves partitioning the data into a specified number of folds, and then the model is trained and evaluated repeatedly on different subsets of the data. Cross validation is a more robust way of evaluating a model's performance and stability; however, it does also increase computational cost as you are training more models based on the number of folds (Müller and

Guido, 2017). In AutoML systems, cross validation can be used for comparing candidate models and different hyperparameter configurations.

Model performance in classification tasks is assessed using metrics like accuracy, F1-score and the area under the receiver operating characteristic curve (ROC-AUC). Accuracy measures the fraction of correctly classified predictions; however, it can create a misleading representation of the model's performance when dealing with datasets that have imbalanced classes, this is because many models tend to bias the majority class (He and Garcia, 2009). The F1-score, which balances precision and recall, and ROC-AUC, which measure how well a model distinguished between positive and negative classes, are widely used metrics for imbalanced classification problems.

Interpretability: Concepts and Definitions

As mentioned in the introduction, there is not a clear consensus on the definition of interpretability or how it should be quantified. Doshi-Velez and Kim (2017) define interpretability with regards to ML as "the ability to explain or to present in understandable terms to a human".

Taxonomies

Approaches for interpretability can be split into different taxonomies; at a high level it can be split into interpretability by design and post-hoc interpretability. Interpretability by design involves training models that are known to be inherently interpretable, while post-hoc approaches involve using interpretability methods after a model has been trained.

Post-hoc methods can be further split into model specific and model agnostic methods. With model agnostic methods further divided into local methods that explain predictions for individual instances in the data, and global methods that focus on how features affect predictions of the whole dataset. Figure 1 shows the relationship between the taxonomy of methods.

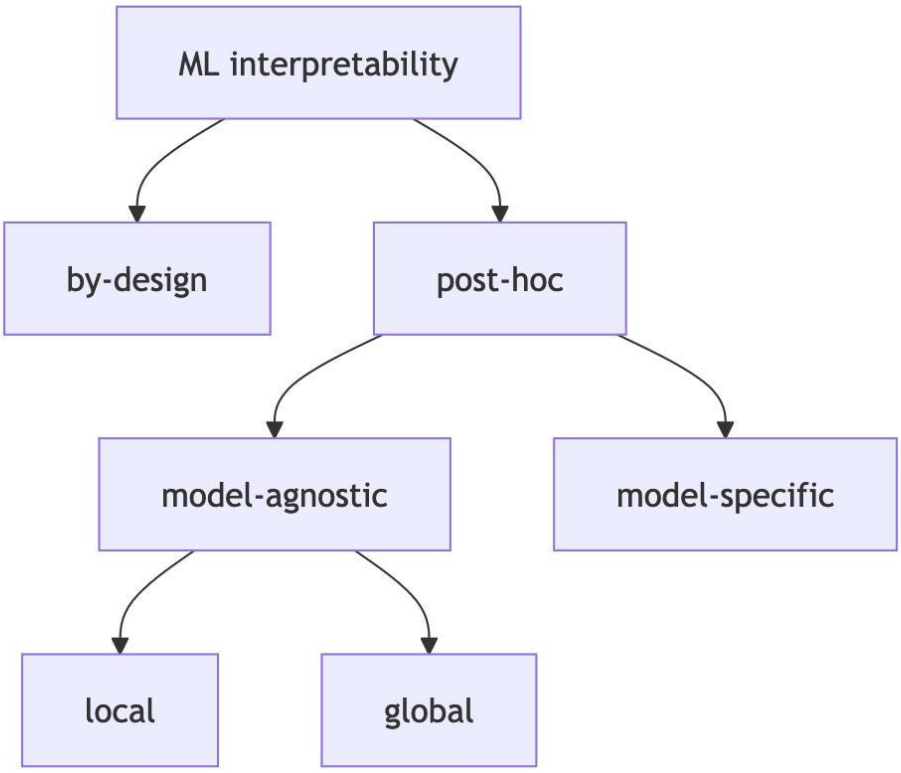

Figure 1: Taxonomy of methods by Molnar (2025).

Interpretability by design

Models that are interpretable by design can be referred to as intrinsically or inherently interpretable. This method focuses on constructing ML systems to be transparent and able to explicitly explain their behaviour based on their inference process (Ai and Narayanan.R, 2021).

Lipton (2018) discusses intrinsic interpretability in the form of transparency and proposes three levels that include: Simulatability (can a person understand the entire model?), decomposability (can individual components of the model be understood?), and algorithmic transparency (is the learning algorithm well understood?). Molnar (2025) adapts this framework into a more practical "scope of interpretability", making the distinction that models can be entirely interpretable, partially interpretable or interpretable at the level of the predictions. This study will focus on the model-level and prediction-level, as algorithmic transparency is consistent across most of the baseline models that are used.

Post-hoc model-agnostic methods

Model-agnostic methods ignore the inner workings of a model, such as learned weights or coefficients, and only analyse how the output predictions of a model changes in relation to

changes to the input feature values, this is done after a model has been trained. Examples of these changes can include perturbation of a feature value.

SIPA is a framework that encompasses the four stages of model-agnostic techniques, these are sampling, intervention, prediction, aggregation. The first stage involves sampling a random subset of the data to reduce computational cost, the second stage intervention involves manipulating the data for input into the model, the prediction stage then creates predictions on the previously manipulated data, the predictions are then aggregated in the fourth stage. Predictions are normally made on each instance that has been sampled, leading to local estimates from the prediction stage, and global estimates result from the aggregation stage (Scholbeck et al., 2020).

Model-specific methods are also applied after a model's training, but their use is restricted to specific machine learning models.

Interpretability Methods and Metrics

Methods

SHAP and Shapley values

The Shapley value comes from cooperative game theory and is a method of assigning payouts to players depending on their marginal contribution to the total payout. Translating this to machine learning and trained models, features can be viewed as the players and the model's prediction value is the payout with Shapley values representing how each feature value contributes to a model's prediction. SHapley Additive exPlanations (SHAP) is a method of explaining feature attributions consistently and locally for predictions of a model, introducing new ways of estimating and aggregating Shapley values across instances for global explanations (Molnar, 2025).

Unlike most explanation methods that are primarily visual or qualitative, SHAP produces numerical feature attributions, enabling quantitative evaluation and comparison, this property making it suitable for integration into AutoML scoring and model selection frameworks.

There are several variants of SHAP that can estimate Shapley values, these include KernelSHAP and Permutation Method for model-agnostic explanations, with TreeSHAP suitable for tree-based ML models (Molnar, 2025).

LIME

Local interpretable model-agnostic explanations (LIME) is an approach for fitting local surrogate models, where an interpretable model is trained to approximate the behaviour of a black-box model in the neighbourhood of a specific instance. The surrogate model provides an explanation for an individual prediction by learning which features are most influential in that local region of the input space.

LIME trains the local surrogate model by selecting an instance of interest to explain, perturbing the instance to generate similar instances, make predictions on the perturbed instances using the original model, weight instances by proximity to original instance, train a simple interpretable model on the weighted instances. The coefficients of the local surrogate model are then used to provide an explanation for the original prediction of that instance (Ribeiro, Singh and Guestrin, 2016; Molnar, 2025).

Metrics

Fidelity

Fidelity is a measure of how accurately an explanation reflects the reasoning and behaviour of the original model to be explained. Ribeiro, Singh and Guestrin (2016) classify local fidelity as an essential criterion for explanations, defining it as an agreement between the predictions of the explanation model and the original model in the vicinity of the instance being explained, utilising LIME to measure and ensure local fidelity across perturbed inputs.

High fidelity indicates that an explanation more precisely identifies the most dominating features of the underlying model, the main idea of fidelity as a metric is that slight modifications to unimportant features should not significantly alter the black-box models predictions (Kundu and Hoque, 2023).

Stability

Stability represents how similar explanations are for similar instances for a fixed model, a high stability signifies that slight variations in feature values does not substantially change the explanation unless it also substantially changes the prediction (Carvalho, Pereira and Cardoso, 2019).

A lack of stability can be caused by high variance or by non-deterministic components of the explanation method used, for example a data sampling step as seen in LIME (Molnar, 2025).

Comprehensibility and Sparsity

Comprehensibility refers to the degree in which an explanation can be understood by a human, it can be influenced by simplicity or size of the explanation, domain knowledge and cognitive load. For example, if an explanation has thousands of features that contribute to its prediction, it is unlikely to be able to be comprehended fully (Ribeiro, Singh and Guestrin, 2016; Molnar, 2025).

Comprehensibility is difficult to measure; however it can be approximated by a quantitative proxy like sparsity, which is implemented and measured on a model by limiting the number of nonzero parameters. By having a smaller number of parameters such as features to analyse, it can make an explanation easier to cognitively understand and yield higher descriptive accuracy (Murdoch et al., 2019).

Global vs Local

Interpretability methods are normally categorised as either local or global, depending on whether the aim is to understand the overall behaviour of a model or the reasoning behind individual predictions. Global explanations describe the average behaviour of a machine learning model across a dataset and what patterns are present in general, while local methods aim to explain individual predictions and higher local interpretability implies understanding the reason for specific decisions (Doshi-Velez and Kim, 2017; Molnar, 2025). Local explanations can be taken for each instance and aggregated to provide global insights of a model, while global methods can also be implemented on subsets of the data to give more regional explanations.

Global interpretability can be important for when scientific understanding or detecting bias is the aim, and local interpretability is useful when decisions made by the model need to be justified (Doshi-Velez and Kim, 2017). There are some benefits of deriving global explanations from local aggregation, mainly that it allows a user to represent global structure while retaining local faithfulness to the original mode, providing more accurate representations of its behaviour. However local methods can suffer from sampling variability and be computationally expensive when aggregated over many instances when compared to purely global methods (Lundberg et al., 2020).

In the context of AutoML , local explanations offer a practical advantage as they can be generated consistently across models and quantitively evaluated using previously mentioned

metrics such as fidelity, stability and sparsity. These can then translate into global explanations of the whole model through aggregation for model ranking.

AutoML and Model Selection

Automated Machine Learning aims to improve the current way of building ML applications by automating the key stages of an ML pipeline, these stages include data preprocessing, model selection and hyperparameter optimisation. Many AutoML frameworks follow a fixed pipeline structure that includes: a linear sequence of data cleaning steps, a feature selection step, one variable preprocessing step and a modelling step. This fixed structure reduces the complexity of the pipeline creation problem, however, can lead to inferior pipeline performance on lesser quality data (Zöller and Huber, 2019).

Combined algorithm selection and hyperparameter optimization (CASH) was introduced by Thornton et al. (2013) as an optimisation problem, in which the selection of algorithm and optimization of its hyperparameters are done simultaneously over a combined search space, and algorithm choice being treated as a hyperparameter. CASH intrinsically couple's algorithm selection and hyperparameter optimisation rather in AutoML rather than traditionally doing these steps sequentially, leading to better performing models. Grid Search, Random Search and Bayesian optimisation are approaches to explore the configuration space for optimization.

AutoML systems predominantly evaluate and rank candidate models using cross-validation to estimate generalisation performance, providing a more reliable estimates than single train-validation splits. Model selection is framed as a performance optimisation problem, where candidate models as well as pipelines are evaluated and ranked using a single performance metric such as accuracy, AUC and loss functions (Zöller and Huber, 2019; Hutter, Kotthoff and Vanschoren, 2019).

Existing AutoML Systems

There are many commercial and open source AutoML frameworks capable of building a complete ML pipeline, some open-source examples include AutoGluon, Auto-sklearn, TPOT and Auto-WEKA.

Auto-WEKA (Thornton et al., 2013) demonstrated that Bayesian optimisation methods could be used to search the combined space of algorithms and hyperparameters to solve the CASH problem, by treating algorithm choice as a categorical hyperparameter in a hierarchical hyperparameter optimization problem.

Auto-sklearn is a tool that create classification and regression pipelines with a semi-fixed structure, with the data cleaning steps being fixed and the preprocessing and modelling steps being selected and tuned via Bayesian optimization technique SMAC. It uses meta-learning to warm start the optimisation process and implements ensemble learning to combine well performing configurations rather than returning a single model (Zöller and Huber, 2019; Feurer *et al.*, 2019).

TPOT was developed by Olson and Moore (2016) as a Tree-based Pipeline Optimisation Tool for building and tuning flexible classification and regression pipelines based on a version of genetic programming. TPOT is essentially a wrapper for scikit-learn with many implementations being from the python machine learning package, except for the XGBoost algorithm.

AutoGluon-Tabular (Erickson et al., 2020) stacks ensembles of multiple models into multiple layers, with experiments showing that this method offers better use of allocated training time than previous methods that focus on finding the best model through hyperparameter searches. Benchmarking was done against other public and commercial AutoML platforms showing that AutoGluon demonstrated superior speed, robustness and accuracy compared to the alternatives.

Despite architectural differences contemporary AutoML systems discussed consistently optimise for a single performance metric, with no focus on interpretability. Auto-sklearn and AutoGluon-Tabular further exacerbate the lack of focus on interpretability by creating ensembles of the models, increasing complexity (Erickson et al., 2020; Thornton et al., 2013; Feurer et al., 2019).

Interpretability in AutoML and Research Gaps

While it is not always needed, interpretability of machine learning models matters as it helps increase user trust in decisions made by models, helps with justification of decisions made in specific instances and ensures fairness and a lack of bias in the underlying model that could cause drastic impact on people's lives (Molnar, 2025).

Existing AutoML frameworks treat model selection as a single-objective optimisation problem, focusing on performance metrics such as accuracy, AUC, F1-score estimated via cross validation (Zöller and Huber, 2019; Hutter, Kotthoff and Vanschoren, 2019). Most interpretability methods, such as LIME and SHAP, are applied after a model has already been selected, rather than influencing the selection process itself. The problem with this is that

AutoML is designed to make machine learning more accessible to users with less expertise, however these users may require more transparency in their application. Incorporating Interpretability into AutoML can help catalyse the adoption of machine learning and AutoML (Molnar, 2025).

Performance-interpretability trade-off

There is an assumption that to achieve higher performing machine learning models, interpretability of the model must be sacrificed, implying a trade-off between interpretability and performance. Freitas (2014) states that many works exploring this trade-off measure comprehensibility in terms of model size, which they think is an over-simplistic view that does not encompass the full picture. Model size can be related to model complexity in this instance, in which case it is fair to argue that black-box models with millions of parameters are not inherently interpretable by humans, however, may be able to be explained via post-hoc methods.

The trade-off has been further challenged by Rudin (2019) who argues that problems involving structured data with meaningful and well represented features can show little difference in performance between complex (black box) classifiers and simpler classifiers. Rudin also argues that interpretable models give the opportunity to interpret the results and reprocess the data iteratively, leading to improvements in performance.

Most of the arguments with regards to whether there is a trade-off come down to a lack of unified definition and quantifiable proxy to measure interpretability as argued for by Doshi-Velez and Kim (2017).

Existing attempts to combine AutoML with XAI methods

Schneider, Bernd Bischl and Thomas (2023) frame model selection as a multi-objective optimisation problem, presenting a model agnostic framework for jointly optimising predictive performance and interpretability. Their approach relies on a complex evolutionary algorithm and proposed bespoke quantification of interpretability in relation to feature sparsity, interaction sparsity of features and sparsity of non-monotone feature effects, these factors limit its translation across to AutoML.

Hasan et al. (2024) combines AutoGluon's AutoML framework and XAI by applying techniques such as SHAP, LIME, integrated gradients and counterfactual analysis to produce and evaluate models for clinical use in diabetes prediction. While this approach illustrates how

explainability techniques can be used to provide model insights, interpretability remains external to the model selection process and are utilised post-hoc.

Khan et al. (2025) implements a partial AutoML pipeline where Optuna is used to automate hyperparameter optimisation for manually selected ensemble models, using SHAP to create post hoc explanations. The use of automated hyperparameter optimisation aligns with the broader definitions of AutoML that include hyperparameter optimisation methods (Zöller and Huber, 2019).

Gaps Identified

While recent work establishes that combining interpretability with automated machine learning is important, existing AutoML frameworks focus on optimising and selecting models based on predictive performance, with interpretability isolated from the model ranking and assessed post hoc (Zöller and Huber, 2019; Carvalho et al., 2019). While there has been propositions for muti-objective approaches to AutoML, they remain complex and hard to generalise (Schneider, Bernd Bischl and Thomas, 2023). As a result, there is a clear research gap in developing a AutoML framework that incorporates quantitative interpretability metrics alongside performance metrics into model ranking and selection.

Section 3: Methodology

Research Design

The aim of this research is to implement interpretability metrics into the scoring and selection of models alongside performance metrics in an automated machine learning pipeline, leading to a proposal for multi-objective ranking. The research adopts an experimental quantitative research design with the development of an AutoML pipeline, comparing a selection of models on different tabular classification datasets and measurements via predefined numerical performance and interpretability metrics.

The hypothesis to be tested are whether interpretability can be integrated into model selection and whether a trade-off between performance and stability exists in this application. The study follows a controlled comparative approach, where supervised classification algorithms are trained and evaluated under consistent preprocessing and optimisation techniques. Model type can be seen as the independent variable in this study, comprising of various classification algorithms including, linear, tree based and ensemble-based methods. The performance and interpretability metrics used to evaluate the framework can be seen as the dependant variables.

Model performance is evaluated using standard classification methods, including accuracy, f1-score and ROC-AUC with the most suitable version of each used whether the problem is a binary or multiclass problem. For interpretability metrics, SHAP was used to create a perturbation-based stability score, measuring consistency of feature attributions under small input variations of features. This stability measure serves as a quantitative indication of explanation robustness for model-agnostic methods.

Stratified Cross-Validation was used for model training and hyperparameter optimization, ensuring evaluation is balanced across class distributions and stable in indicating generalization performance. Following the hyperparameter selection, models are then retrained on the full training set and evaluated on a held-out test set to ensure unbiased performance estimates. Interpretability metrics are then also calculated using a subset of the test instances using the PermutationExplainer method from the SHAP package as it is model agnostic.

To combine both performance and interpretability metrics, a multi-objective ranking function is defined, each metric is normalised to a comparative scale between 0 and 1 and weighted

according to a predefined trade-off parameter. This function enables systematic comparison of models under different performance-interpretability trade-off requirements.

All experiments are conducted in a controlled computational environment with a fixed random seed of 42 to ensure reproducibility.

Datasets

Dataset selection criteria

A mixture of tabular classification datasets were used to implement and evaluate the automated ML framework, this was done to try and encompass and compare how it performed on data with varying characteristics. A mixture of binary and multiclass datasets were used, some with balanced classes while others had different levels of imbalance to simulate real-world predictive modelling instances.

The y labels were encoded for each test using the scikit-learn LabelEncoder package, ensuring that the labels are encoded similarly for each dataset and y labels are a 1D NumPy array for integration into metric calculations. Each dataset was split into training and test sets with an 80/20% split respectively using stratified sampling to preserve class proportions.

Dataset 1: Telco Customer Churn

The Telco Customer churn dataset used in this study was obtained from a Kaggle repository (blastchar, 2017) originally provided as a sample dataset by IBM. This dataset contains information about a fictional telco company providing home phone and internet services, indicating whether a customer has stayed or left within the last month.

The dataset consists of 7043 instances with 20 input features in total, 3 being numerical and the remaining 17 being categorical. The target variable represent whether a customer has churned or not, this is indicated with the labels “yes” and “no” which would be encoded to values of 0 or 1 for a binary classification problem. As with most churn classification problems, the classes are imbalanced, with 26.5% of the instance being a customer that has churned.

Dataset 2: Wine Quality

The wine quality dataset was sourced from the UCI Machine Learning Repository (Cortez et al., 2009) and imported via the ucimlrepo python package. It contains data on Portuguese “Vinho Verde” wine samples, with the goal to be to model wine quality from a scale of 0-10 based on physicochemical measurement data.

The dataset consists of 6497 instances with 11 numerical input features. The target variable represents a quality score of the wine on a scale of 0-10, present in the dataset is 7 unique scores between 3 and 9 indicating a multiclass classification problem.

The classes are moderately imbalanced with majority of the samples being classified as normal wines (scores of 5 and 6) than excellent or poor quality. This class imbalance means the dataset is suitable for evaluating model performances and the framework under realistic imbalance conditions.

Dataset 3: Breast Cancer Wisconsin (Diagnostic)

The Breast Cancer Wisconsin dataset was sourced from the UCI Machine Learning Repository (Street, Wolberg and Mangasarian, 1993). It contains tabular data that describe characteristics present in a digitized image of a breast mass with classification on whether the mass is benign or malignant.

The dataset consists of 569 instances with 29 numerical input features. The target variable represents whether a breast mass is malignant or benign with the letters M or B correspondingly, these were encoded to values of 0 for benign and 1 for malignant. The classes are relatively balanced with 357 benign instances and 212 malignant instances

Dataset 4: Statlog (German Credit Data)

The Statlog dataset was also sourced from the UCI Machine Learning Repository (Hofmann, 1994). It contains data with features to classify if a person is a good or bad credit risk. The dataset contains 1000 instance with 7 numerical and 13 categorical input features. The target variable was initially encoded as 1 = good credit risk and 2 = bad credit risk however with label encoder these were transformed again to 0 and 1 respectively. The classes are also moderately imbalanced with 700 instances for good credit risk and 300 instances for bad credit risk.

Dataset 5: Glass Identification

The Glass Identification dataset was the last dataset sourced from the UCI Machine Learning Repository (German, 1989). It is a multiclass classification dataset from the USA Forensic Science Service in which 6 classifications of glass are defined in terms of their oxide content. It contains only 214 instances with 9 numerical feature inputs. The classes also have imbalances with the instance of class label of "tableware" only accounting for 4.2% of the data compared to the 35.5% of the most common class.

Preprocessing pipeline

Data preprocessing was standardised across datasets, with the assumption that data input into the pipeline is already clean and to a good quality, the datasets used in this study contained no missing values after initial inspection, therefore no imputation strategy was required. The pipeline class from scikit learn was used to sequentially apply preprocessing transformations prior to model fitting, ensuring consistent and leakage-free processing (Müller and Guido, 2017).

```
pipeline = Pipeline([
    ("features", feature_grouper_cls(**feature_grouper_kwargs)),
    ("preprocess", preprocessor),
    ("model", model),
])
```

Figure 2: Section of code showing the pipeline steps.

Feature Handling

Feature handling is the first step in the pipeline, with initial processing being completed using a custom GroupFeatures class. The purpose of this stage is to identify the data types of each feature in the dataset and group them into a list of numerical and categorical features for subsequent transformation.

Once categorical features were identified from features that include datatypes of object, string or category, they were then examined for high cardinality. Features with more than 20 unique categories were removed to prevent overfitting, reduce dimensional expansion related to one-hot encoding and lower computational cost (Géron, 2019).

```python
# Create a transformer class to group features in and drop columns

from sklearn.base import BaseEstimator, TransformerMixin

class GroupFeatures(BaseEstimator, TransformerMixin):

    # Constructor method
    def __init__(self, max_unique=20):
        self.max_unique = max_unique

    #Fit method learns column groupings and detects high cardinality
    def fit(self, X, y=None):
        categorical = X.select_dtypes(include=["object", "string",
        "category"])
        numeric = X.select_dtypes(include="number")
        boolean = X.select_dtypes(include="bool")

        #Detects high cardinality above maximum value specified
        high_cardinality = categorical.nunique() > self.max_unique

        #Creates numerical column list
        self.numeric_columns_ =
        numeric.columns.union(boolean.columns).tolist()
        #Creates categorical column list - high cardinality features
        self.categorical_columns_ =
        high_cardinality[~high_cardinality].index.tolist()

        self.keep_columns_ = self.numeric_columns_ + self.categorical_columns_

        return self

    # passes through the data untransformed
    def transform(self, X):
        return X[self.keep_columns_]
```

Figure 3: GroupFeatures class for feature handling.

Feature transformation

The second step of the pipeline transforms the numerical and categorical features. Categorical features were transformed using one-hot encoding creating a binary numeric feature for each unique category in that feature. One-hot encoding is good for nominal categorical data and is safer to use as implying order when not needed is likely to confuse the learning algorithm (Andriy Burkov, 2019). As high cardinality was filtered in the previous feature handling step, this does not lead to an excessive expansion in dimensionality.

Z-score standardization was applied to all numerical features to ensure comparable scale across predictors. Scaling is important for models that are distance-based or that utilise gradient descent, which are sensitive to feature magnitude, while tree-based models are generally not affected by scaling (Müller and Guido, 2017; Géron, 2019).

```
# ColumnTransformer for feature preprocessing
from sklearn.compose import make_column_selector

preprocessor = ColumnTransformer(
    transformers=[
        ("num", StandardScaler(),
        make_column_selector(dtype_include="number")),
        ("cat", OneHotEncoder(handle_unknown="ignore"),
        make_column_selector(dtype_include=["object", "string",
        "category"])),
    ],
    remainder="drop"
).set_output(transform="default")
```

Figure 4: Column transformer for preprocessing of specific feature types.

Cross-Validation

StratifiedKFold cross-validation with 3 folds was used to preserve class distribution across training and validation splits, which is important for imbalanced classification problems (He and Garcia, 2009). A fixed random seed of 42 was used to ensure reproducibility across experiments. All preprocessing and model fitting steps being embedded into the scikit-learn pipeline ensured transformations such as feature handling were learned exclusively on training folds to prevent data-leakage (Müller and Guido, 2017).

Model selection

Models

The candidate models used in this study were the classification versions of Logistic Regression, Decision Trees, Random Forest and LightGBM. This encompasses a diversity of model types varying in complexity such as linear model, single tree, ensemble trees and boosted models. Only four models were selected due to time constraints and minimisation of computational cost. Each model was initialised with the random state of 42, logistic regression was set to have max_iter of 5000 and both Random Forest and LightGBM were set to have n_jobs = -1 to utilise all CPU cores for efficiency.

```
# Define models
models = {
    "logreg": LogisticRegression(max_iter=5000, random_state=SEED),
    "dt": DecisionTreeClassifier(random_state=SEED),
    "rf": RandomForestClassifier(n_jobs=-1, random_state=SEED),
    #"svm": SVC(probability=True), --> taken out due to speed of the
    experiments
    "lgb": LGBMClassifier(n_jobs=-1, random_state=SEED),
}
```

Figure 5: Dictionary of models for each experimental run.

Hyperparameter Optimisation

Hyperparameter Optimisation was conducted on each candidate model using RandomizedSearchCV, with stratified cross validation over 3 folds and 20 hyperparameter sample iterations. The optimisation was evaluated using accuracy, f1_weighted and roc_auc_ovr metrics which were selected so that they could work on both multiclass and binary classification problems. The model was refit on the best parameters for roc_auc_ovr.

RandomizedSearchCV was selected over Grid Search due to it being more computationally efficient and suitable for a limited search budget (Bergstra and Bengio, 2012). Bayesian optimisation methods were not adopted to maintain transparency of the search process and reduce additional algorithmic complexity.

Model selection workflow

The full workflow of the automated model selection framework is as follows:

1. Perform cross-validation hyperparameter optimisation on each candidate model.
2. Select best hyperparameters.
3. Retrain the model on the full training set with the best hyperparameters.
4. On the test split of the data compute the final performance metrics and SHAP stability and sparsity.
5. Apply all computed metrics into the ranking function, ROC AUC was used as the performance metric even though others were computed.

Performance Evaluation

Predictive performance was evaluated on the held-out test set using ROC AUC One-vs-Rest as the primary metric for ranking, accuracy and weighted F1-score were also calculated for

descriptive comparison. Mean cross-validation performance during hyperparameter optimisation was also saved to assess model generalisation stability if needed. The choice of metrics was made due to their applicability to binary and multiclass classification problems.

```
from sklearn.metrics import accuracy_score, f1_score, roc_auc_score

def evaluate_model_on_test(best_pipeline, X_test, y_test):
    y_pred = best_pipeline.predict(X_test)
    y_proba = best_pipeline.predict_proba(X_test)

    metrics = {
        "accuracy": accuracy_score(y_test, y_pred),
        "f1_weighted": f1_score(y_test, y_pred, average="weighted"), # keeping
        multiclass metrics
    }

    #print("Unique classes in y_test:", np.unique(y_test))
    #print("Shape of y_proba:", y_proba.shape)
    #print("Classes learned by model:",
    best_pipeline.named_steps["model"].classes_)

    # ROC-AUC handling binary and multiclass
    n_classes = len(best_pipeline.named_steps["model"].classes_)

    if n_classes == 2:
        metrics["roc_auc"] = roc_auc_score(y_test, y_proba[:, 1]) # Binary
    else:
        metrics["roc_auc"] = roc_auc_score(y_test, y_proba, multi_class="ovr",
         average="macro") # Multiclass

    return metrics, y_pred, y_proba
```

Figure 6: Function for model evaluation on test set.

SHAP Stability

SHAP Method

Model interpretability was assessed using SHAP (Shapley Additive exPlanations), which is a model agnostic framework based on Shapley values from cooperative game theory (Lundberg and Lee, 2017). SHAP provides additive feature attributions that quantify how each feature in the dataset contributes to a model's predictions.

SHAP's PermutationExplainer was selected for this study due to its mode-agnostic compatibility across the candidate models which consisted of linear, tree-based and boosted models. Despite it being a slower method, and there being faster TreeSHAP methods for tree-based model, the same method was to be used across models to ensure consistency in SHAP

calculations. PermutationExplainer estimates Shapley values through iterative feature permutation of the inputs and model evaluation, providing local explanations for individual predictions (Molnar, 2025).

A background dataset was sampled from the training data and used to construct the SHAP masker, ensuring explanations were computed relative to representative training data.

Stability definition

To quantify explanation robustness, a perturbation-based stability metric was defined. For a given instance, small perturbations were applied to the numerical features and then SHAP values were recomputed for each perturbed instance (Alvarez-Melis and Jaakkola, 2018). Cosine similarity was then computed between the original SHAP value and each perturbed SHAP value and averaged over all perturbations for each instance.

Let $\phi(x)$ denote the SHAP attribution vector for an instance x, then $\phi\left(x^{(k)}\right)$ denote the SHAP attribution vector for the Kth perturbed instance. Then stability is defined as:

$$Stability(x) = \frac{1}{K}\sum_{k=1}^{K} sim\left(\phi(x), \phi\left(x^{(k)}\right)\right)$$

Where similarity was computed using cosine similarity:

$$sim(a, b) = \frac{a \cdot b}{\| a \| \| b \|}$$

Similarity scores were scaled to the range between 0 and 1. For multiclass problems, SHAP values were averaged across classes before similarity computation.

Numerical features were perturbed by adding Gaussian noise proportional to the features training sets standard deviation, while categorical features were not changed to maintain their meaning. Stability was calculated per instance in the sample taken from the test data and then averaged across all instances to obtain a model-level stability score while managing computational cost.

Computational Setting

To ensure computational feasibility and explanation reliability the following parameters were implemented for all datasets analysed using this method:

- Background sample size: 100

- Test instances explained per model: 200
- Number of perturbations per instance: 30
- Noise level: 2% of standard deviation of training data
- SHAP maximum evaluations: 200
- Random seed: 42

All computation were performed on a local CPU-based environment. This configuration balances explanation fidelity with computation and run time constraints.

SHAP Sparsity

While stability is a measure of explanation robustness, sparsity captures the comprehensibility of the explanations. Cognitive complexity is another dimension of interpretability and explanations that involve fewer dominant features are generally easier for humans to understand and interpret (Doshi-Velez and Kim, 2017; Molnar, 2025).

For SHAP explanations, sparsity indicates how concentrated predictive contribution is among a smaller set of features rather than being distributed across many features which would lead to higher complexity.

Sparsity Definition

A SHAP attribution vector of instance x with p features can be represented by:

$$\phi(x) = (\phi 1, \phi 2, \dots, \phi p)$$

First absolute contributions are computed, and the total attribution magnitude of the attribution vector is calculated.

$$T(x) = \sum_{j=1}^{p} |\phi_j(x)|$$

To quantify sparsity, the cumulative contribution of the top-k most influential features, with the largest absolute SHAP values, is calculated and then divided by the total attribution magnitude.

$$S_k(x) = \frac{\sum_{j \in Top-k} |\phi_j(x)|}{T(x) + \epsilon}$$

Where ϵ represents a small constant to prevent division by zero.

For multiclass problems, SHAP values were averaged across classes before calculating sparsity. And the final sparsity score was obtained by averaging over all instances in the test sample.

The sparsity metric is defined as the proportion of the total SHAP contribution that is captured by the top-k most influential features for a given prediction. A higher value of the sparsity metric indicates that prediction contribution is concentrated among fewer features leading to improved interpretability via comprehensibility, whereas a lower value indicates that contribution is distributed among many features.

Excessive sparsity can also reflect an oversimplified decision structure on the model's part, therefore pairing this jointly with stability and other metrics is needed in an interpretability aware model ranking framework.

Multi-Objective Ranking

Once each metric had been calculated they were then fed into a multi-objective ranking function, each metric should have been normalised when calculated so that values were comparable and between 0 and 1. Metrics being on the same scale ensures one metric does not bias the outcome of the ranking score.

First the interpretability metrics needed to be combined into an interpretability score with their scale kept between 0 and 1. This score was defined as:

$$Interpretability = \beta \cdot Stability + (1 - \beta) \cdot Sparsity$$

where β is the interpretability ratio with a value between 0 and 1, giving the user a way to control the importance of stability vs sparsity.

The Interpretability score was then input into the final ranking score of a similar form, defined as:

$$Score = \alpha \cdot Performance + (1 - \alpha) \cdot Interpretability$$

where performance represents the ROC-AUC one-vs-rest metric evaluated on the test set, interpretability is the interpretability score, and the performance ratio α controls the trade-off between performance and interpretability. $\alpha = 1$ would mean that model selection is based on performance only, with an $\alpha < 0.5$ if the purpose was for interpretability-based ranking.

The baseline configuration used for the experiments were with $\alpha = 0.5$ for balance ranking, however ranking decisions can be evaluated post-hoc with different values of the performance ratio as every metric was saved per experiment. A linear weighted sum was selected for both scoring formulas as it provides an intuitive and controllable trad-off mechanism that a user can easily manipulate for their use case.

```
# define scoring function
def multi_objective_score(performance, shap_stability, shap_sparsity,
    perf_ratio=0.5, interp_ratio = 0.5):
    assert 0 <= performance <= 1
    assert 0 <= shap_stability <= 1
    assert 0 <= shap_sparsity <= 1
    assert 0 <= perf_ratio <= 1
    assert 0 <= interp_ratio <= 1

    interp = interp_ratio * shap_stability + (1 - interp_ratio) *
    shap_sparsity

    return perf_ratio * performance + (1-perf_ratio) * interp
```

Figure 7: Function for implementing multi-objective scoring.

Reproducibility and implementation

All experiments were implemented using python version 3.11.14, utilising libraries for machine learning such as scikit-learn, SHAP and LightGBM. Experiments were run on a local machine equipped with an AMD Ryzen 7 2700X Eight-Core Processor and 16 GB of installed RAM. Random seeds were fixed to 42 throughout the notebook as well as the global seed of the notebook to ensure reproducibility of experiments. StratifiedKFold cross validation was used consistently with 3 folds across experiments and all preprocessing, and model training steps were embedded into pipelines to prevent data-leakage. The full environment configuration for experiments was saved to a yaml file and is provided in the projects GitHub repository.

Ethical Considerations

For this study all datasets used were publicly available online via Kaggle or the UCI Machine Learning Repository. No human participants were involved, and no personal identifiable data was collected or stored.

Section 4: Results

Experimental Overview

The interpretability aware AutoML pipeline was used on 5 datasets to train and rank 4 different models per dataset, using Logistic Regression, Decision Tree, Random Forest and LightGBM algorithms. Once trained in the pipeline, performance metrics of Accuracy, weighted F1-score and ROC AUC OVR were calculated, quantitative interpretability metrics SHAP stability a SHAP sparsity were also approximated on a sample of 200 test instances from each dataset.

Stability and Sparsity were then combined into a composite interpretability metric with an interpretability ratio (β) to control the trade-off between stability and sparsity. Interpretability was then fed into a similar multi-objective formula with ROC AUC as the performance metric and a performance ratio (α) to create a balanced multi-objective score for each model to rank them according to both performance and interpretability. Both α and β were set to 0.5 for a balanced ranking function.

As all metrics were saved for each model created for each dataset, sensitivity of the performance ratio (α) could also be analysed, this was done for the Wine Quality dataset. This section will show the results seen for each model created using each dataset and compare the main performance metric ROC AUC with composite interpretability, as well as compare how SHAP Stability and SHAP Sparsity vary per model.

Dataset Level Results

Telco Customer Churn

The metrics presented in Table 1 indicate the that the Decision Tree classifier ranked highest with a multi-objective ranking score of 0.897 and Random Forest ranked the lowest with a score of 0.805. Predictive performance, measured by ROC AUC, showed minimal variation across models, with values differing by a maximum of 0.012, however LightGBM achieved the highest value with an ROC AUC of 0.843. In contrast, mean sparsity demonstrated substantial variation, ranging from Decision Tree (0.931) and Random Forest (0.543), mirroring the behaviour of the composite ranking score.

Table 1: Performance and interpretability metrics for Telco Customer Churn dataset.

model	accuracy	f1_weighted	roc_auc	mean_stability	mean_sparsity	ranking_score
dt	0.792	0.778	0.831	0.997	0.931	0.897
lgb	0.752	0.764	0.843	0.995	0.692	0.843
logreg	0.795	0.790	0.838	0.997	0.651	0.831
rf	0.799	0.790	0.839	0.998	0.543	0.805

The composite interpretability score was compared to the predictive performance (ROC AUC), as illustrated in Figure 8. While the range of ROC AUC is relatively smaller than that of the interpretability scores, an inverse relationship can be seen for Decision Tree, Logistic Regression and Random Forest. LightGBM does not follow this trend, achieving the highest predictive performance while having the second-highest interpretability score.

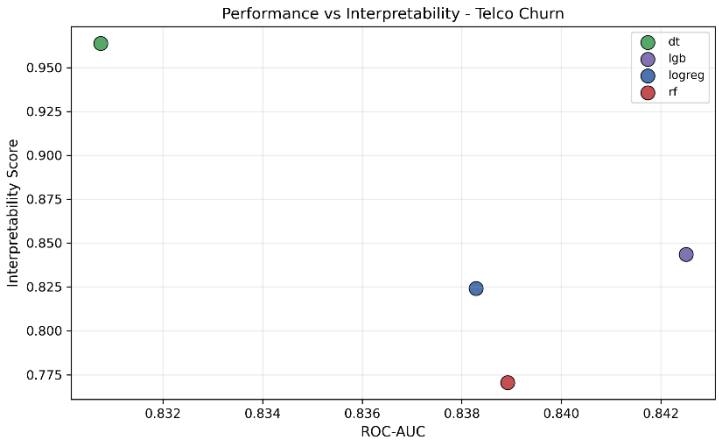

Figure 8: Composite interpretability of Stability and Sparsity compared with performance metric ROC AUC for the Telco Customer Churn dataset, interpretability ratio = 0.5.

Both Stability and Sparsity metrics were compared in Figure 9, stability values are tightly clustered (0.995-0.998) indicating robust explanations across models. As previously mentioned, sparsity exhibits higher variation, suggesting that differences in interpretability were driven primarily by the concentration of feature attribution rather than perturbation robustness for this dataset.

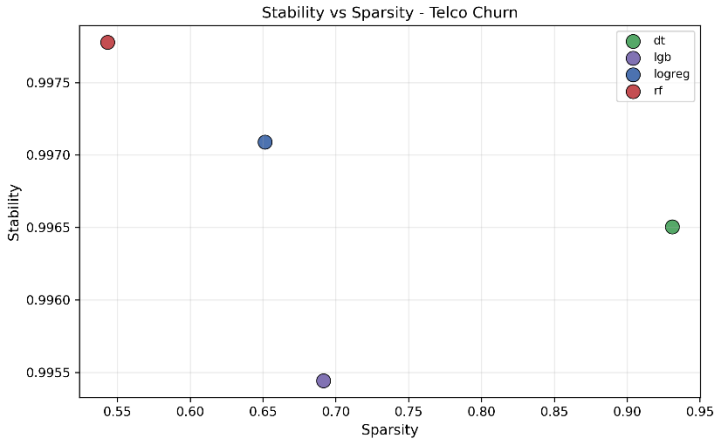

Figure 9: Comparison of Stability and Sparsity for the Telco Customer Churn dataset.

Breast Cancer Wisconsin (Diagnostic)

In the Breast Cancer Wisconsin **dataset,** Table 2 **indicates** that with a score of 0.979 the Decision Tree model ranked highest in the multi-objective ranking score, and Logistic Regression ranking the lowest with a score of 0.877. Contrasting the results of the ranking score, Decision Tree ranked the lowest in predictive performance with an ROC AUC value of 0.96, Random Forest and Logistic Regression jointly ranked the highest with a score of 0.997.

Mean sparsity demonstrated substantial variance again with a score of 1.000 for Decision Tree and 0.533 for Logistic Regression. It is worth noting that the sparsity score for Decision Tree indicates that the model makes 100% of its decisions based on the top 5 features, it is possible the prediction contribution is concentrated among even fewer features.

Table 2: Performance and interpretability metrics for Breast Cancer Wisconsin (Diagnostic) dataset.

model	accuracy	f1_weighted	roc_auc	mean_stability	mean_sparsity	ranking_score
dt	0.939	0.938	0.960	0.994	1.000	0.979
lgb	0.965	0.965	0.995	0.897	0.661	0.887
rf	0.974	0.973	0.997	0.993	0.540	0.882
logreg	0.965	0.965	0.997	0.980	0.533	0.877

Comparing the composite interpretability score with interpretability ratio = 0.5 between stability and sparsity, Figure 10 shows a clustering of LightGBM, Random Forest and Logistic Regression in the bottom right corner. For these 3 models' performance and interpretability barely changes. Decision Tree stands on its own in the top left corner, with a variation in performance compared to the other models of ~0.045, while it varies on a larger scale in terms

of interpretability by ~ 0.2. It shows a weak performance interpretability trade-off and indicates ranking largely depends on interpretability differences in this dataset.

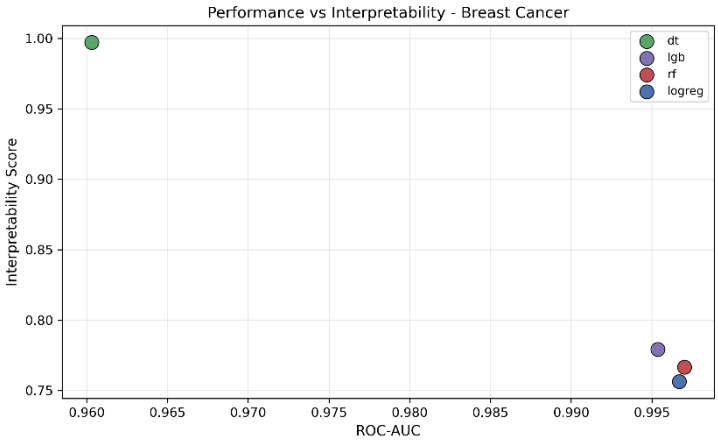

Figure 10: Composite interpretability of Stability and Sparsity compared with performance metric ROC AUC for the Breast Cancer Wisconsin dataset, interpretability ratio = 0.5.

Stability of the 3 clustered models was stable as shown in Figure 11, whereas LightGBM showed noticeably lower stability compared to the other models. Decision Tree has clearly much higher sparsity than the other models, even if it has similar stability to Random Forest and Logistic Regression, this indicates interpretability separation is primarily driven by sparsity.

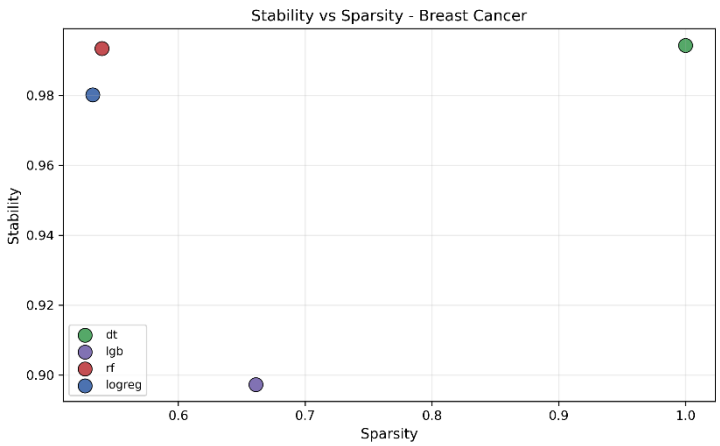

Figure 11: Comparison of Stability and Sparsity for the Breast Cancer Wisconsin dataset.

Statlog (German Credit Data)

Table 3 indicates Decision Tree achieved the highest multi-objective ranking score of 0.811, while Logistic Regression obtained the lowest at 0.760. Predictive performance of ROC AUC was scored highest by the Logistic Regression model and ranged between 0.808 (highest) to 0.664 (lowest), indicating a moderate variation in performance across models. Mean sparsity showed a substantial variation again with Decision Tree had a value of 0.939 while Logistic Regression had a value of 0.425. Mean stability exhibits a smaller range from 1.00 to 0.978.

Table 3: Performance and interpretability metrics for Statlog (German Credit Data).

model	accuracy	f1_weighted	roc_auc	mean_stability	mean_sparsity	ranking_score
dt	0.660	0.667	0.664	0.978	0.939	0.811
lgb	0.750	0.731	0.762	0.992	0.597	0.778
rf	0.765	0.767	0.804	0.997	0.501	0.777
logreg	0.775	0.757	0.808	1.000	0.425	0.760

Figure 12 illustrates the relationship between predictive performance and interpretability. Random Forest and Logistic regression clustered close with performance ~ 0.8 and interpretability score of ~ 0.73. Decision Tree shows a strong separation from the other models with interpretability > 0.95 and performance much lower. The scatter plot suggests an inverse relationship between the metrics, as performance increases interpretability decreases.

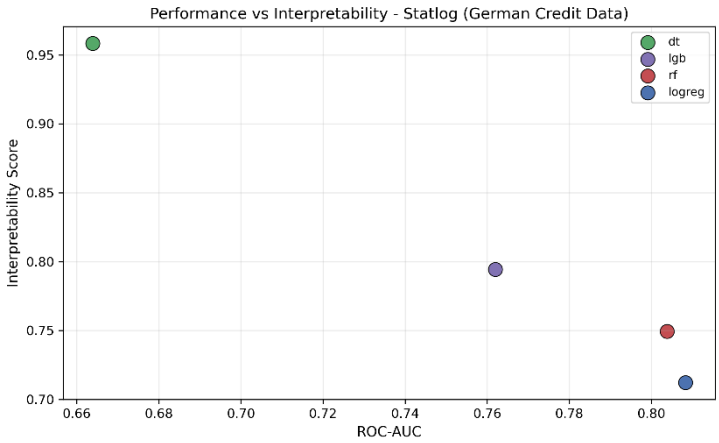

Figure 12: Composite interpretability of Stability and Sparsity compared with performance metric ROC AUC for the Statlog dataset, interpretability ratio = 0.5.

Figure 13 shows the high variance in sparsity and a lower variance in stability, stability ranging from ~ 1.00 for Logistic regression and ~ 0.98 for Decision Tree. There is a clearly observed inverse relationship with higher sparsity associated with a lower stability.

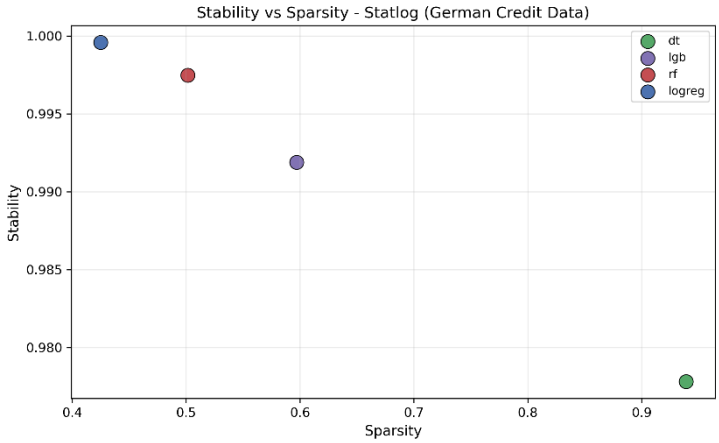

Figure 13: Comparison of Stability and Sparsity for the Statlog dataset.

Wine Quality

For the Wine Quality dataset Table 4 show the Decision Tree model once again ranked the highest in multi-objective ranking with a score of 0.863, while Logistic Regression had the lowest score of 0.838. In contrast to the ranking score the Decision Tree model ranked the lowest in predictive performance with an ROC AUC score of 0.732, the model with the highest ROC AUC was LightGBM with a score of 0.866.

Table 4: Performance and interpretability metrics for Wine Quality dataset.

model	accuracy	f1_weighted	roc_auc	mean_stability	mean_sparsity	ranking_score
dt	0.521	0.500	0.732	0.991	0.995	0.863
rf	0.607	0.604	0.845	0.991	0.757	0.859
lgb	0.692	0.680	0.866	0.960	0.727	0.855
logreg	0.535	0.506	0.785	0.991	0.792	0.838

Figure 14 shows a slight inverse relationship between the composite interpretability score and predictive performance, although not completely linear. The variance of both axis are moderate, Decision Tree being the most separate with an interpretability score ~ 1.00, the rest of the models fit between an interpretability score of 0.84 and 0.90.

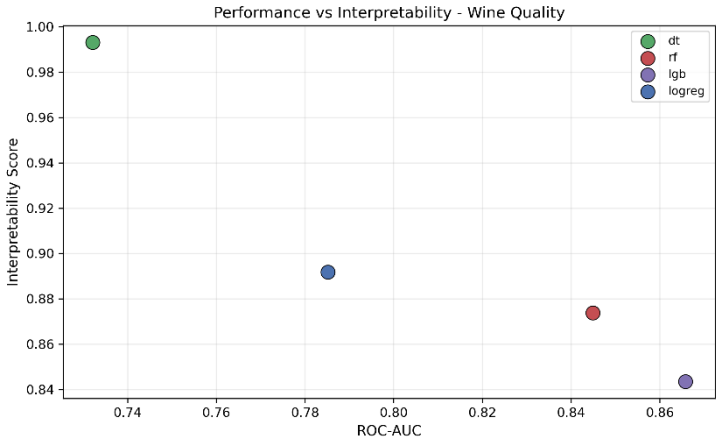

Figure 14: Composite interpretability of Stability and Sparsity compared with performance metric ROC AUC for the Wine Quality dataset, interpretability ratio = 0.5.

When comparing sparsity and stability in Figure 15, it can be seen that Logistic Regression, Random Forest and Decision Tree are clustered at a stability of ~ 0.99 while LightGBM shows to be less robust with a stability of 0.96. The variance in sparsity is much larger than that of stability, indicating that sparsity is contributing more to interpretability separation in this case.

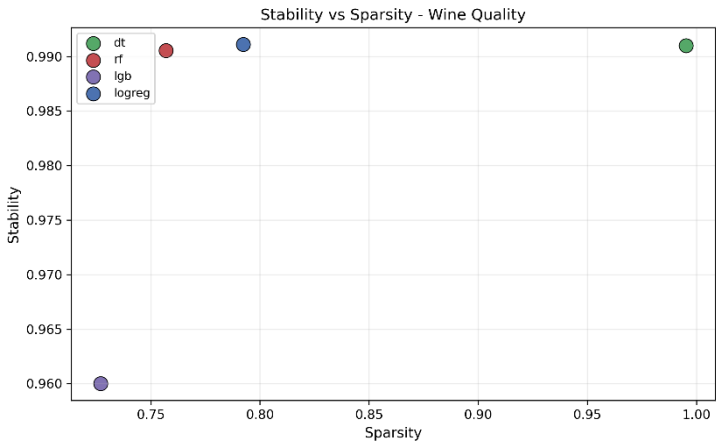

Figure 15: Comparison of Stability and Sparsity for the Wine Quality dataset.

Glass Identification

Table 5 indicates that for the glass identification dataset the Random Forest model ranked the highest in the multi-objective ranking score with a value of 0.941, with Logistic Regression ranking the lowest with a score of 0.897. Random Forest also ranked the highest in predictive performance with an ROC AUC of 0.966 with Decision Tree having the lowest score of 0.862. All models showed high robustness with scores ranging from 0.991 for Decision Tree and 0.986 for Logistic Regression.

Table 5: Performance and interpretability metrics for Glass Identification dataset.

model	accuracy	f1_weighted	roc_auc	mean_stability	mean_sparsity	ranking_score
rf	0.791	0.784	0.966	0.998	0.834	0.941
lgb	0.651	0.644	0.946	0.989	0.823	0.926
dt	0.744	0.727	0.862	0.991	0.913	0.907
logreg	0.628	0.643	0.895	0.986	0.813	0.897

The Decision Tree model is shown to again how the lowest ROC AUC while having the highest interpretability score in Figure 16. The interpretability variance has a range of ~ 0.06 and the ROC AUC varies from 0.86 to 0.97. Other than the Decision Tree model, the other models seem to have a slight positive trend of ROC AUC and interpretability with Random Forest scoring the second highest in interpretability and the highest in predictive performance.

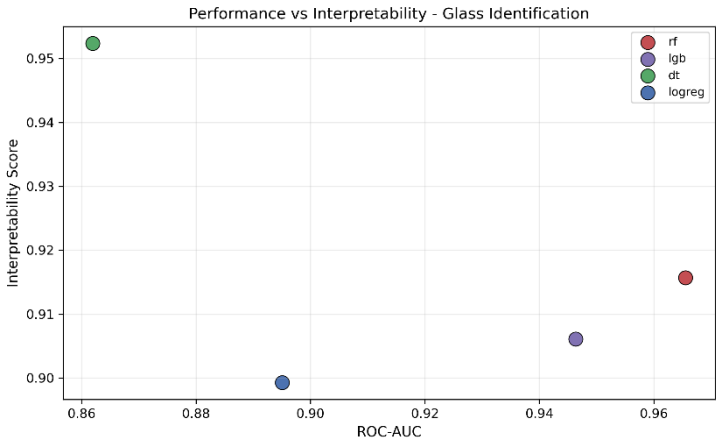

Figure 16: Composite interpretability of Stability and Sparsity compared with performance metric ROC AUC for the Glass Identification dataset, interpretability ratio = 0.5.

Figure 17 shows all models have high robustness with a low dispersion of stability with a range of ~ 0.015, however sparsity has higher variance indicating higher contribution to interpretability score.

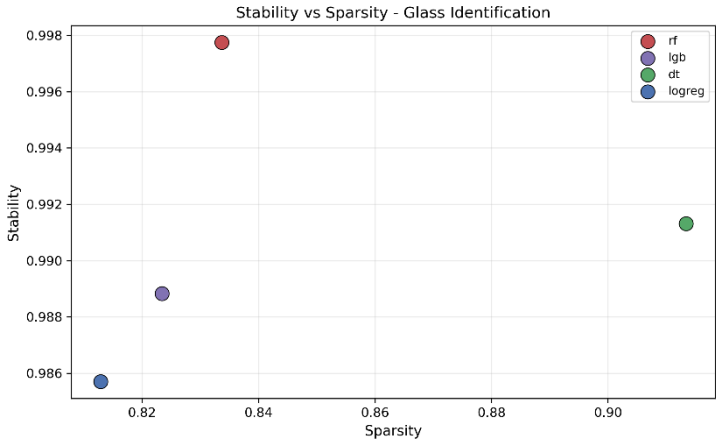

Figure 17: Comparison of Stability and Sparsity for the Glass Identification dataset.

Sensitivity to performance ratio (α)

The performance ratio in the multi-objective ranking formula for this study helps the user define the trade-off they want to specify between model ranking, with a performance ratio of 0 meaning the ranking will solely focus on interpretability metrics when ranking models, and a value of 1 means a sole focus on performance metrics in when ranking the models.

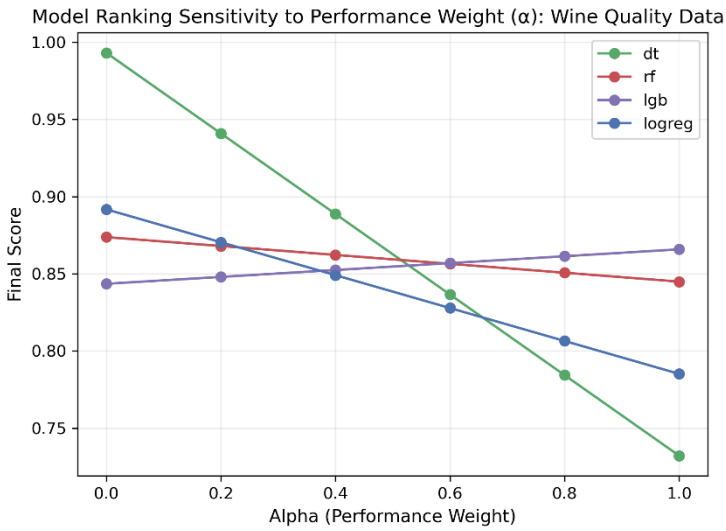

Figure 18: Sensitivity of multi-objective ranking score to performance ratio α for Wine Quality Dataset.

To evaluate the dependence of model ranking on the performance ratio α, sensitivity analysis was conducted on the wine quality dataset. The model ranking score for each model was recalculated while varying the value of α from 0 to 1 in steps of 0.2, with the interpretability ratio $\beta = 0.5$ keeping an equal balance for interpretability between SHAP sparsity and SHAP stability. The dataset was selected to evaluate this due to noticeable variation in both predictive

performance and interpretability on similar scales as seen in Figure 7, this provides a suitable case for examining the behaviour.

Figure 18 illustrates how ranking score varies across values of α. When α = 0 and the focus is on ranking based on interpretability alone, Decision Tree achieved the highest-ranking score. Alternatively, for α = 1 and ranking is focused on only predictive performance, LightGBM achieves the highest-ranking score. The ranking of the models cross-over at a value of $\alpha \approx 0.6$ indicating that model selection within this interpretability aware AutoML pipeline is sensitive to the weighting assigned by the user on how much focus they want on predictive performance.

Section 5: Discussion

Summary of key findings

Comparing the composite interpretability score with ROC AUC in the results section for each dataset shows a typical trend indicating a trade-off between model performance and interpretability. Although some models did not conform to this behaviour resulting in higher performance and interpretability that other models, seen in analysis of the Glass Identification, Breast Cancer and Telco Customer Churn datasets.

It was seen that model ranking can be influenced under changes in the performance ratio (α), specifically seen in the Wine Quality dataset, where the rankings were completely flipped when looking at a ranking score focused on only predictive performance or only interpretability.

Decision Tree was consistently shown to be the most interpretable model with the lowest predictive performance. The behaviour of Logistic Regression, LightGBM and Random Forest varied more in both metrics. For some datasets like Glass Identification and Statlog (German Credit Data) Logistic Regression ranked the lowest in the defined composite interpretability score.

For most datasets, all models exhibited high SHAP stability with low variance between the models, with values ranging from 0.98 to 1.00. For the Breast Cancer and Wine Quality datasets, LightGBM showed the biggest drop in stability with 0.90 and 0.96 respectively.

SHAP sparsity varied much more than stability, in most cases ranging from a 0.5-1.0 with noticeable separation between the models. Decision Tree consistently showed to be more sparse across datasets, LightGBM showed to rank second in sparsity across 3 of the 5 datasets while Logistic Regression and Random Forest show less sparsity. The data shows that sparsity does not strictly increase or decrease with inherent model complexity across datasets.

Performance vs Interpretability Trade-off

A trade-off between the composite Interpretability and Performance is observed in most datasets, whereas the learning performance improves the interpretability score decreases. This behaviour is seen most clearly in analysis of the models created for the Wine Quality and Statlog (German Credit Data) dataset, while the same trend can be observed in the other three datasets.

It is important to note that in all cases the Decision Tree model skews the trend, and for the Breast Cancer, Glass Identification and Telco customer Churn datasets the trend is not obvious. Logistic regression, Random Forest and LightGBM cluster in the Breast cancer dataset, performing similarly in predictive performance and interpretability.

For the glass identification dataset, without Decision Tree, the other models show a more positive trend with Random Forest having the highest predictive performance and second highest (after Decision Tree) measure interpretability. Similarly, in the Telco Customer Churn dataset LightGBM shows the same behaviour of having the highest predictive performance while having strong interpretability based on the composite interpretability score.

The data seen aligns with the case study results of (Atrey et al., 2025) where the general trend is to see that learning performance improves with decreasing interpretability, however the relationship is not monotonic and there are instance where interpretable models can outperform less interpretable models in prediction performance.

Role of Stability and Sparsity

For most datasets stability of every model is seen to be generally high and tightly clustered in the range of 0.98-1.00, with the exception of LightGBM for the Wine Quality and Breast Cancer datasets. This may be attributable to the fact these datasets had the most numerical features and no categorical features, SHAP perturbations were conducted on only numerical features, however for these same datasets, the other model types maintained high stability.

Sparsity on the other hand had much more variation in its values for each model, in cases such as the Statlog (German Credit Data) this value ranged from ~ 0.4-0.95. As the composite interpretability score was equally balanced between sparsity and stability, in all experiments it seems sparsity drives the interpretability separation of models and that interpretability variation is not uniform across metrics.

The clear difference in variance and resulting differences in how sparsity and stability drive the proposed composite interpretability score, reinforces the notion highlighted by Doshi-Velez and Kim (2017) that there is not a unified definition or quantifiable proxy to measure interpretability, as well as the difficulties in using quantifiable proxies such as sparsity for model comparison of interpretability.

It is also seen that SHAP sparsity does not align with preconceived concepts of model complexity, where ensembles such as Random Forest and LightGBM are considered more

complex due to their internal complexity compared with Logistic Regression (Haddouchi and Berrado, 2024). While Decision Tree consistently ranked the highest, LightGBM consistently ranked second in terms of sparsity. This shows that interpretability cannot be determined solely by model family in this context and that interpretability can vary depending on dataset and use case.

User Preference α Sensitivity

It is seen that ranking of models can change as the performance ratio of the multi-objective ranking formula changes for the Wine Quality dataset with a crossover point of $\alpha \approx 0.6$, however some model ranking changes also occur between $\alpha \approx 0.2$-0.6. Generally, with $\alpha < 0.6$ the model selection is interpretability focused, while $\alpha > 0.6$ shows more focus on predictive performance. This crossover behaviour indicates that model ranking and selection is sensitive to the performance ratio (α), this demonstrates that the most suitable model for a user will be indicated by the ranking score based on a user's preference selection, with the designed AutoML pipeline allowing selection to reflect these priorities.

Implications for AutoML

Traditional AutoML systems focus on model selection a single-objective problem, focusing on predictive performance metrics such as accuracy or ROC AUC (Zöller and Huber, 2019). Interpretability is often treated separately as a post-hoc explanation technique and is not directly involved in model selection. As a result of this, the selection process rarely incorporates interpretability criteria into optimisation and ranking procedures, and outputs a model based solely on a predictive performance.

The findings of this study indicate that incorporating interpretability in the form of explanation robustness (stability) and feature attribution concentration (sparsity) into model selection can significantly impact model ranking. Sensitivity analysis shows that varying the performance ratio (α) can shift the model rankings, indicating that the optimal model is not always fixed and is dependent on the importance assigned to predictive performance or interpretability.

These results suggest that multi-objective model selection is a meaningful extension to conventional AutoML pipelines. By including interpretability metrics into the ranking formula, a user can align model choice with their priorities whether that is interpretability focus or predictive performance. The lightweight AutoML pipeline implemented in this study

demonstrates that this integration is computationally feasible for tabular classification tasks without the requirement of a complex optimisation framework.

Limitations

While the proposed pipeline demonstrates meaningful behaviour and results across multiple tabular binary and multiclass classification datasets, some limitations should be acknowledged to contextualise the findings.

To meet computational constraints and ensure reproducibility, some evaluation limitations were implemented. First three-fold stratified cross-validation was implemented with 20 hyperparameter sample iterations on a limited search space, by increasing the number of hyperparameters in the search, or increasing iterations and folds, better and more robust performance metrics may have been achieved.

The study was also restricted to four model families of varying complexity: Logistic Regression, Decision Tree, Random Forest and LightGBM. The study did not involve Neural Networks or some other families such as K-Nearest Neighbours or Support Vector Machines, as a result of this, conclusions regarding performance-interpretability trade-off may not generalise to all other models.

Interpretability was quantified using SHAP-based stability and sparsity metrics derived from PermutationExplainer. While this approach ensures consistency across different models, SHAP explanations themselves are based on an approximation from a sample of 200 test instances, which leads to the possibility of results being sensitive to the sampling of test data or background instances in every use case.

The analysis was limited to tabular classification tasks and tested on datasets with a maximum of 7043 instance a 30 input features. The behaviour seen for this study may differ for regression problems, high dimensionality data domains and deep learning applications. Therefore, the findings in this study should be interpreted within the scope of its application to tabular classification tasks rather than a universal conclusion across machine learning tasks.

Lastly, interpretability is known to be multi-dimensional, while this study explored quantification of stability and sparsity using SHAP, it would be worth exploring other interpretability metrics in unison to see how they affect the outcome of model ranking and efficiency of the AutoML pipeline. These limitations open the scope for future study in different domains and further applications.

Section 6: Conclusion

The aims of this dissertation were to investigate and explore the feasibility of incorporating interpretability into Automated Machine Learning model selection, as well as investigating quantitative measures of interpretability and whether a trade-off would be evident with predictive performance.

It was found that including interpretability in the model selection process can significantly impact model selection, and models selected only based on predictive performance may not be the most optimal solution if the user needs interpretability considerations. A general trade-off between predictive performance and the defined composite interpretability metric was seen across 5 tabular classification datasets, however this behaviour was not completely monotonic and there were some cases where models outperformed others in both performance, measure by ROC AUC one-vs-rest, and interpretability. There was not a clear relationship between robustness of explanations (stability) and feature attribution concentration (sparsity); throughout the experiments, stability was shown to consistently be high across models with low variance, while sparsity showed more separation. It was seen that sparsity drove the value of the composite interpretability metric more than stability, translating to it having a bigger impact on the ranking scores.

This study implemented a lightweight multi-objective AutoML pipeline that incorporated interpretability into model ranking and selection. A composite interpretability metric was proposed using SHAP explanations to quantify stability and sparsity, which was then translated to global approximation for model ranking. It was then shown that the proposed multi-objective ranking function demonstrated preference-sensitive ranking that a user can control via a performance ratio (α) to indicate the most relevant model for their needs. It was also shown that the relationship between performance and interpretability varied across datasets, indicating multi-objective ranking behaviour is also dataset dependent.

The results of this study show that interpretability can be integrated into AutoML in the case of tabular classification problems in binary and multiclass settings, with multi-objective ranking being feasible in the same settings and reflecting a user's priorities. While narrow in scope and being more akin to a proof of concept, further development in this area could help democratise the use of Machine Learning in even more areas that require more interpretable models.

Some future areas to explore include broadening the scope of model classes included and analysed in the AutoML pipeline, these could include SVM, KNN or rule-based models to examine if similar behaviour occurs. Expanding this application into other machine learning areas such as regression, deep learning and big data. Alternative quantitative measures of interpretability could be explored and defined, ideally model-agnostic metrics and methodologies to try and create a unified and measurable definition of interpretability. Lastly further investigations into more computationally efficient strategies, such a model-specific SHAP variants like TreeSHAP, could improve scalability.

References

Ai, Q. and Narayanan.R, L. (2021) ‘Model-agnostic vs. Model-intrinsic Interpretability for Explainable Product Search’, *Proceedings of the 30th ACM International Conference on Information & Knowledge Management* [Preprint]. Available at: https://doi.org/10.1145/3459637.3482276.

Alvarez-Melis, D. and Jaakkola, T.S. (2018) ‘On the Robustness of Interpretability Methods’, *arxiv.org* [Preprint]. Available at: https://doi.org/10.48550/arXiv.1806.08049.

Andriy Burkov (2019) *THE HUNDRED-PAGE MACHINE LEARNING BOOK*. Andriy Burkov.

Atrey, P. *et al.* (2025) *Demystifying the Accuracy-Interpretability Trade-Off: A Case Study of Inferring Ratings from Reviews*, *arXiv.org*. Available at: https://arxiv.org/abs/2503.07914 (Accessed: 22 February 2026).

Bergstra, J. and Bengio, Y. (2012) ‘Random Search for Hyper-Parameter Optimization’, *Journal of Machine Learning Research*, 13, pp. 281–305. Available at: https://www.jmlr.org/papers/volume13/bergstra12a/bergstra12a.pdf.

Biran, O. and Cotton, C. (2017) ‘Explanation and justification in machine learning: A survey’, pp. 8–13. Available at: https://www.cs.columbia.edu/~orb/papers/xai_survey_paper_2017.pdf (Accessed: 4 January 2026).

blastchar (2017) *Telco Customer Churn*, *www.kaggle.com*. Available at: https://www.kaggle.com/datasets/blastchar/telco-customer-churn (Accessed: 22 January 2026).

Carvalho, D.V., Pereira, E.M. and Cardoso, J.S. (2019) ‘Machine Learning Interpretability: A Survey on Methods and Metrics’, *Electronics*, 8(8), p. 832. Available at: https://doi.org/10.3390/electronics8080832.

Cortez, P. *et al.* (2009) ‘Modeling wine preferences by data mining from physicochemical properties’, *Decision Support Systems*, 47(4), pp. 547–553. Available at: https://doi.org/10.1016/j.dss.2009.05.016.

Doshi-Velez, F. and Kim, B. (2017) ‘Towards A Rigorous Science of Interpretable Machine Learning’, *arXiv:1702.08608 [cs, stat]*, 2(2). Available at: https://arxiv.org/abs/1702.08608.

Erickson, N. *et al.* (2020) ‘AutoGluon-Tabular: Robust and Accurate AutoML for Structured Data’, *arXiv:2003.06505 [cs, stat]* [Preprint]. Available at: https://arxiv.org/abs/2003.06505.

Feng, P. *et al.* (2024) *Mastering AI: Big Data, Deep Learning, and the Evolution of Large Language Models -- AutoML from Basics to State-of-the-Art Techniques*, *arXiv.org*. Available at: https://arxiv.org/abs/2410.09596v2 (Accessed: 4 January 2026).

Feurer, M. *et al.* (2019) ‘Auto-sklearn: Efficient and Robust Automated Machine Learning’, *Automated Machine Learning*, pp. 113–134. Available at: https://doi.org/10.1007/978-3-030-05318-5_6.

Freitas, A.A. (2014) ‘Comprehensible classification models’, *ACM SIGKDD Explorations Newsletter*, 15(1), pp. 1–10. Available at: https://doi.org/10.1145/2594473.2594475.

GDPR (2018) *General Data Protection Regulation (GDPR)*, *GDPR*. Available at: https://gdpr-info.eu/.

German, B. (1989) *UCI Machine Learning Repository*, *archive.ics.uci.edu*. Available at: https://archive.ics.uci.edu/dataset/42/glass+identification.

Géron, A. (2019) *Hands-on machine learning with Scikit-Learn and TensorFlow concepts, tools, and techniques to build intelligent systems*. 2nd edn. O’Reilly Media, Inc.

Haddouchi, M. and Berrado, A. (2024) *A survey and taxonomy of methods interpreting random forest models*, *arXiv.org*. Available at: https://arxiv.org/abs/2407.12759.

Hasan, R. *et al.* (2024) ‘Towards Transparent Diabetes Prediction: Combining AutoML and Explainable AI for Improved Clinical Insights’, *Information*, 16(1), p. 7. Available at: https://doi.org/10.3390/info16010007.

He, H. and Garcia, E.A. (2009) ‘Learning from Imbalanced Data’, *IEEE Transactions on Knowledge and Data Engineering*, 21(9), pp. 1263–1284. Available at: https://doi.org/10.1109/tkde.2008.239.

He, X., Zhao, K. and Chu, X. (2021) ‘AutoML: A survey of the state-of-the-art’, *Knowledge-Based Systems*, 212, p. 106622. Available at: https://doi.org/10.1016/j.knosys.2020.106622.

Hofmann, H. (1994) *UCI Machine Learning Repository*, *archive.ics.uci.edu*. Available at: https://archive.ics.uci.edu/dataset/144/statlog+german+credit+data.

Hutter, F., Kotthoff, L. and Vanschoren, J. (2019) *Automated Machine Learning*, *The Springer Series on Challenges in Machine Learning*. Edited by F. Hutter, L. Kotthoff, and J. Vanschoren. Springer International Publishing. Available at: https://doi.org/10.1007/978-3-030-05318-5.

Khan, M.S. *et al.* (2025) 'Explainable AutoML models for predicting the strength of high-performance concrete using Optuna, SHAP and ensemble learning', *Frontiers in Materials*, 12. Available at: https://doi.org/10.3389/fmats.2025.1542655.

Lipton, Z.C. (2018) 'The mythos of model interpretability', *Communications of the ACM*, 61(10), pp. 36–43. Available at: https://doi.org/10.1145/3233231.

Lundberg, S.M. *et al.* (2020) 'From local explanations to global understanding with explainable AI for trees', *Nature Machine Intelligence*, 2(1), pp. 56–67. Available at: https://doi.org/10.1038/s42256-019-0138-9.

Lundberg, S.M. and Lee, S.-I. (2017) *A Unified Approach to Interpreting Model Predictions*, *Neural Information Processing Systems*. Curran Associates, Inc. Available at: https://papers.nips.cc/paper_files/paper/2017/hash/8a20a8621978632d76c43dfd28b67767-Abstract.html.

Miller, T. (2019) 'Explanation in artificial intelligence: Insights from the social sciences', *Artificial Intelligence*, 267, pp. 1–38. Available at: https://doi.org/10.1016/j.artint.2018.07.007.

Molnar, C. (2025) *Interpretable machine learning*. 3rd edn. Available at: https://christophm.github.io/interpretable-ml-book.

Müller, A.C. and Guido, S. (2017) *Introduction to Machine Learning with Python: A Guide for Data Scientists*. O'reilly.

Murdoch, W.J. *et al.* (2019) 'Definitions, methods, and applications in interpretable machine learning', *Proceedings of the National Academy of Sciences*, 116(44), pp. 22071–22080. Available at: https://doi.org/10.1073/pnas.1900654116.

Olson, R.S. and Moore, J.H. (2016) ‘TPOT: A Tree-based Pipeline Optimization Tool for Automating Machine Learning’, *PMLR*, pp. 66–74. Available at: https://proceedings.mlr.press/v64/olson_tpot_2016.html.

Ribeiro, M.T., Singh, S. and Guestrin, C. (2016) ‘“Why Should I Trust You?”: Explaining the Predictions of Any Classifier’, *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining - KDD ’16*, pp. 1135–1144. Available at: https://doi.org/10.1145/2939672.2939778.

Ripan Kumar Kundu and Khaza Anuarul Hoque (2023) ‘Explainable Predictive Maintenance is Not Enough: Quantifying Trust in Remaining Useful Life Estimation’, *Proceedings of the Annual Conference of the Prognostics and Health Management Society*, 15(1). Available at: https://doi.org/10.36001/phmconf.2023.v15i1.3472.

Rudin, C. (2019) ‘Stop Explaining Black Box Machine Learning Models for High Stakes Decisions and Use Interpretable Models Instead’, *Nature Machine Intelligence*, 1(5), pp. 206–215. Available at: https://doi.org/10.1038/s42256-019-0048-x.

Schneider, L., Bernd Bischl and Thomas, J. (2023) ‘Multi-Objective Optimization of Performance and Interpretability of Tabular Supervised Machine Learning Models’, *Proceedings of the Genetic and Evolutionary Computation Conference*, pp. 538–547. Available at: https://doi.org/10.1145/3583131.3590380.

Scholbeck, C.A. *et al.* (2020) ‘Sampling, Intervention, Prediction, Aggregation: A Generalized Framework for Model-Agnostic Interpretations’, *Communications in Computer and Information Science*, pp. 205–216. Available at: https://doi.org/10.1007/978-3-030-43823-4_18.

Street, W.N., Wolberg, W.H. and Mangasarian, O.L. (1993) ‘Nuclear feature extraction for breast tumor diagnosis’, *Biomedical Image Processing and Biomedical Visualization* [Preprint]. Edited by R.S. Acharya and D.B. Goldgof. Available at: https://doi.org/10.1117/12.148698.

Thornton, C. *et al.* (2013) ‘Auto-WEKA: Combined Selection and Hyperparameter Optimization of Classification Algorithms’, *arXiv:1208.3719 [cs]* [Preprint]. Available at: https://arxiv.org/abs/1208.3719.

Zöller, M.-A. and Huber, M.F. (2019) *Survey on Automated Machine Learning*, *arXiv.org*. Available at: https://arxiv.org/abs/1904.12054.

Appendices

A: Experiments and Code

All code and experiments for this project can be found in the allocated GitHub repository by following the link below. The notebook used to run experiment was Final_AutoML_Pipeline_experiments.ipynb.

https://github.com/MunroRoss/MSc-Simple-Multi-Objective-AutoML-Pipeline